

Лабораторная работа №5

1 Характеристики системы управления памятью

- Общий объем оперативной памяти:
980308 KB
- Объем раздела подкачки (swap):
233928 KB
- Размер страницы виртуальной памяти:
4096 байт
- Объем свободной физической памяти в ненагруженной системе:
609840 KB
- Объем доступной физической памяти в ненагруженной системе:
659240 KB
- Объем свободного пространства в разделе подкачки:
69488 KB

2 Эксперимент №1

Последняя строка файла `report.log`:

10000000

2.1 Аварийное завершение процесса `mem.bash`

После запуска скрипт `mem.bash` был аварийно завершен операционной системой. Для анализа причины завершения были просмотрены последние записи системного журнала, связанные с данным процессом.

```
karasik@vds2923400:~/os_lab5$ sudo dmesg | grep "mem.bash" | tail -n 2
[769981.171681] oom-kill:constraint=CONSTRAINT_NONE,nodemask=(null),cpuset=user.slice,mems_allowed=0,global_oom,task_memcg=/user.slice/user-1002.slice/session-484.scope,task=mem.bash,pid=1711186,uid=1002
[769981.171759] Out of memory: Killed process 1711186 (mem.bash) total-vm:819872kB, anon-rss:725496kB, file-rss:0kB, shmem-rss:0kB, UID:1002 pgtables:1636kB oom_score_adj:0
```

Из приведенной записи видно, что процесс `mem.bash` был завершен механизмом OOM Killer. Причиной завершения стала нехватка памяти в системе, о чем свидетельствует сообщение:

Out of memory: Killed process 1711186 (mem.bash)

На момент аварийного завершения процесса были зафиксированы следующие параметры:

- идентификатор процесса: 1711186;
- общий объем виртуальной памяти процесса: 819872 KB;
- объем анонимной резидентной памяти: 725496 KB;
- объем памяти, связанной с файловыми отображениями: 0 KB;
- объем разделяемой памяти: 0 KB;
- объем памяти, занятой таблицами страниц: 1636 KB;
- значение `oom_score_adj`: 0.

2.2 Наблюдение за процессом `mem.bash` с помощью утилиты `top`

Таблица 1: Изменение параметров памяти системы по данным `top`

Время	Mem free, MiB	Mem used, MiB	buff/cache, MiB	Swap free, MiB	Swap used, MiB	avail Mem, MiB
16:57:12	473.3	202.0	282.0	75.1	153.3	614.7
16:57:17	265.3	409.9	282.1	75.1	153.3	406.8
16:57:22	64.6	610.6	282.1	75.1	153.3	206.1
16:57:27	69.7	794.3	93.4	66.9	161.6	45.9
16:57:29	71.0	827.0	59.3	3.3	225.2	30.3
16:57:31	52.5	867.4	37.5	0.0	228.4	0.8
16:58:04	740.4	143.1	73.8	25.1	203.3	707.1

Из таблицы видно, что в процессе работы скрипта объем свободной памяти уменьшался, а объем используемой памяти увеличивался.

Таблица 2: Изменение параметров процесса `mem.bash` по данным `top`

Время	VIRT, KB	RES, KB	SHR, KB	%CPU	%MEM
16:57:12	34208	26980	3128	93.8	2.8
16:57:17	246992	240028	3128	93.8	24.5
16:57:22	451592	444628	3128	93.8	45.4
16:57:27	652892	645556	2888	88.2	65.9
16:57:29	751628	685200	2844	81.2	69.9
16:57:31	795584	723276	380	24.0	73.8

По мере роста массива процесс продолжал оставаться среди верхних процессов по потреблению ресурсов.

Ближе к моменту аварийного завершения в верхних строках списка процессов появился процесс `kswapd0`. Это связано с тем, что система начала активно работать с механизмами подкачки при нехватке свободной оперативной памяти. В момент времени 16:57:31 `kswapd0` находился выше `mem.bash` в списке процессов, а свободное пространство в `swap` было равно 0.0 MiB.

После аварийного завершения процесса `mem.bash` в списке процессов он отсутствовал. В верхних строках остались другие системные процессы.

2.3 Записи системного журнала после наблюдения в top

После исчезновения процесса `mem.bash` из списка процессов были просмотрены последние записи системного журнала, относящиеся к скрипту.

Из системного журнала видно, что процесс `mem.bash` был завершён механизмом OOM Killer. На момент завершения были зафиксированы следующие параметры:

```
karasik@vds2923400:~/os_lab5$ sudo dmesg | grep "mem.bash" | tail -n 2
[774999.775769] oom-kill:constraint=CONSTRAINT_NONE,nodemask=(null),cpuset=user.slice,mems_allowed=0,global_oom,task_memcg=/user.slice/user-1002.slice/session-491.scope,task=mem.bash,pid=1723068,uid=1002
[774999.775840] Out of memory: Killed process 1723068 (mem.bash) total-vm:800732kB, anon-rss:728064kB, file-rss:0kB, shmem-rss:0kB, UID:1002 pgtables:1604kB oom_score_adj:0
```

- идентификатор процесса: 1723068;
- общий объем виртуальной памяти процесса: 800732 KB;
- объем анонимной резидентной памяти: 728064 KB;
- объем памяти, связанной с файловыми отображениями: 0 KB;
- объем разделяемой памяти: 0 KB;
- объем памяти, занятой таблицами страниц: 1604 KB;
- значение `oom_score_adj`: 0.

значение в последней строке файла `report.log`: 10000000

2.4 Второй этап

был создан скрипт `run_mem.sh`, запускающий оба скрипта в фоновом режиме.

Таблица 3: Изменение параметров памяти системы при работе двух процессов

Время	Mem free, MiB	Mem used, MiB	buff/cache, MiB	Swap free, MiB	Swap used, MiB	avail Mem, MiB
18:08:55	504.9	188.5	263.9	41.1	187.3	627.0
18:08:56	447.6	245.8	264.0	41.1	187.3	569.6
18:08:58	387.3	306.1	264.0	41.1	187.3	509.3
18:09:55	53.9	870.3	33.1	0.0	228.4	0.3
18:10:45	743.5	115.4	98.5	11.3	217.1	722.6

Таблица 4: Изменение параметров процессов `mem.bash` и `mem2.bash`

Время	Процесс	VIRT, KB	RES, KB	SHR, KB	%CPU	%MEM
18:08:55	mem.bash	32492	25252	2980	44.4	2.6
18:08:55	mem2.bash	32360	25072	3072	44.4	2.6
18:08:56	mem.bash	61928	54556	2980	43.8	5.6
18:08:56	mem2.bash	61664	54640	3072	37.5	5.6
18:09:55	mem2.bash	792548	759300	672	24.0	77.5

В начале эксперимента оба процесса находились в верхних строках списка процессов `top`. Они имели близкие значения `VIRT`, `RES` и `%MEM`, так как выполняли одинаковые действия

Позднее процесс `mem.bash` был завершён раньше, после чего в списке процессов остался только `mem2.bash`

2.5 Записи системного журнала для двух процессов

После завершения обоих процессов были просмотрены записи системного журнала, относящиеся к скриптам `mem.bash` и `mem2.bash`.

```
karasik@vds2923400:~/os_lab5$ sudo dmesg | grep -E 'Out of memory: Killed process [0-9]+ ' | tail -n 2
[779299.809332] Out of memory: Killed process 1733101 (mem.bash) total-vm:408560kB, anon-rss:381056kB, file-rss:0kB, shmem-rss:0kB, UID:1
002 pgtables:832kB oom_score_adj:0
[779360.733441] Out of memory: Killed process 1733102 (mem2.bash) total-vm:804032kB, anon-rss:770052kB, file-rss:0kB, shmem-rss:0kB, UID:
1002 pgtables:1616kB oom_score_adj:0
```

На момент аварийного завершения процесса `mem.bash` были зафиксированы следующие параметры:

- идентификатор процесса: 1733101;
- общий объем виртуальной памяти процесса: 408560 KB;
- объем анонимной резидентной памяти: 381056 KB;
- объем памяти, связанной с файловыми отображениями: 0 KB;
- объем разделяемой памяти: 0 KB;
- объем памяти, занятой таблицами страниц: 832 KB;
- значение `oom_score_adj`: 0.

На момент аварийного завершения процесса `mem2.bash` были зафиксированы следующие параметры:

- идентификатор процесса: 1733102;
- общий объем виртуальной памяти процесса: 804032 KB;
- объем анонимной резидентной памяти: 770052 KB;
- объем памяти, связанной с файловыми отображениями: 0 KB;
- объем разделяемой памяти: 0 KB;
- объем памяти, занятой таблицами страниц: 1616 KB;
- значение `oom_score_adj`: 0.

Также были зафиксированы последние строки файлов отчетов:

```
report.log:
5000000
```

```
report2.log:
10000000
```

2.6 Вывод по второму этапу

При одновременном запуске двух экземпляров скрипта память начала расходоваться быстрее, чем при запуске одного процесса. Оба процесса увеличивали свои массивы независимо друг от друга, поэтому суммарное потребление оперативной памяти росло быстрее.

Процесс `mem.bash` был завершен первым. После его завершения часть памяти была освобождена, поэтому процесс `mem2.bash` смог продолжить работу и увеличить массив до большего размера. Однако затем и он был завершен механизмом `OOM Killer`, так как доступная оперативная память и пространство в разделе подкачки были практически исчерпаны.

Таким образом, эксперимент показал, что при одновременной работе нескольких процессов, активно выделяющих память, аварийное завершение наступает быстрее, чем при работе одного процесса. Также было видно, что система активно использовала механизм подкачки, что подтверждается появлением процесса `kswapd0` среди верхних процессов и полным заполнением `swap`.

3 Обработка результатов

В данном разделе приведены графики изменения наблюдаемых величин и краткие выводы по каждому из них.

3.1 Пороговые значения размера массива

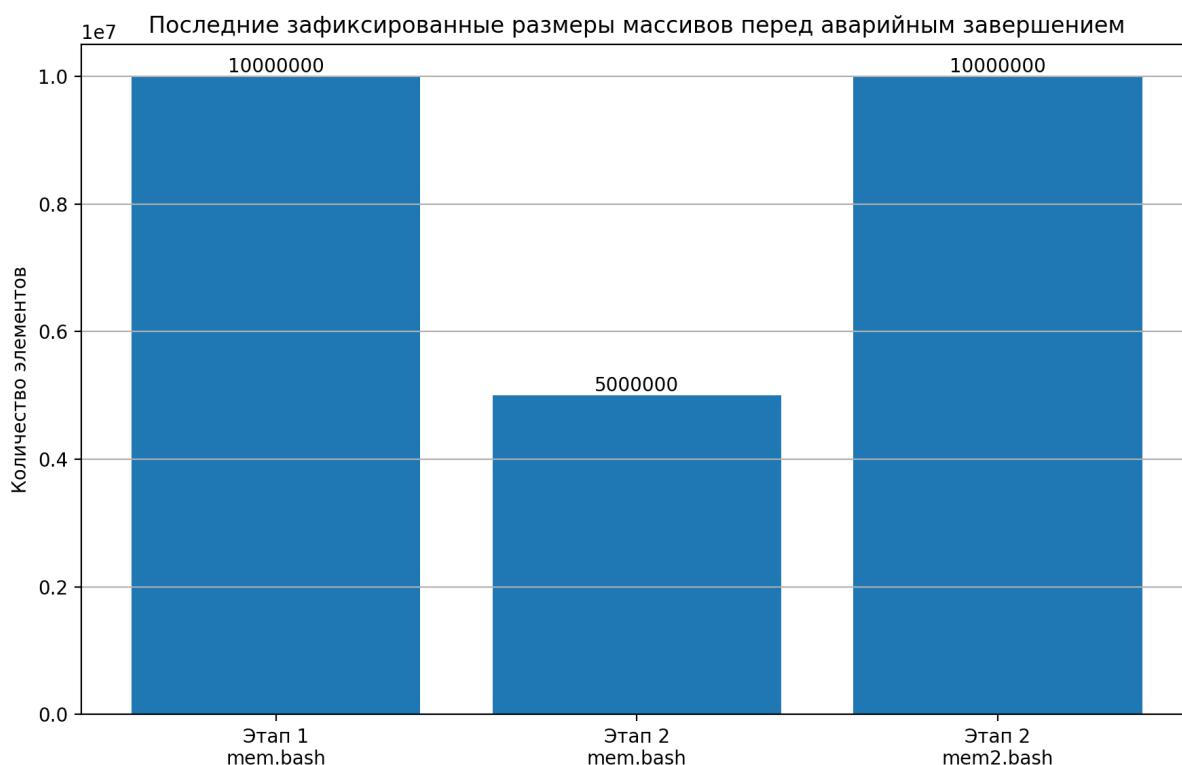


Рис. 1: Последние зафиксированные размеры массивов перед аварийным завершением

Вывод. Последнее зафиксированное значение размера массива составило 10000000 элементов для одиночного запуска, 5000000 элементов для `mem.bash` во втором этапе и 10000000 элементов для `mem2.bash`.

3.2 Один процесс mem.bash

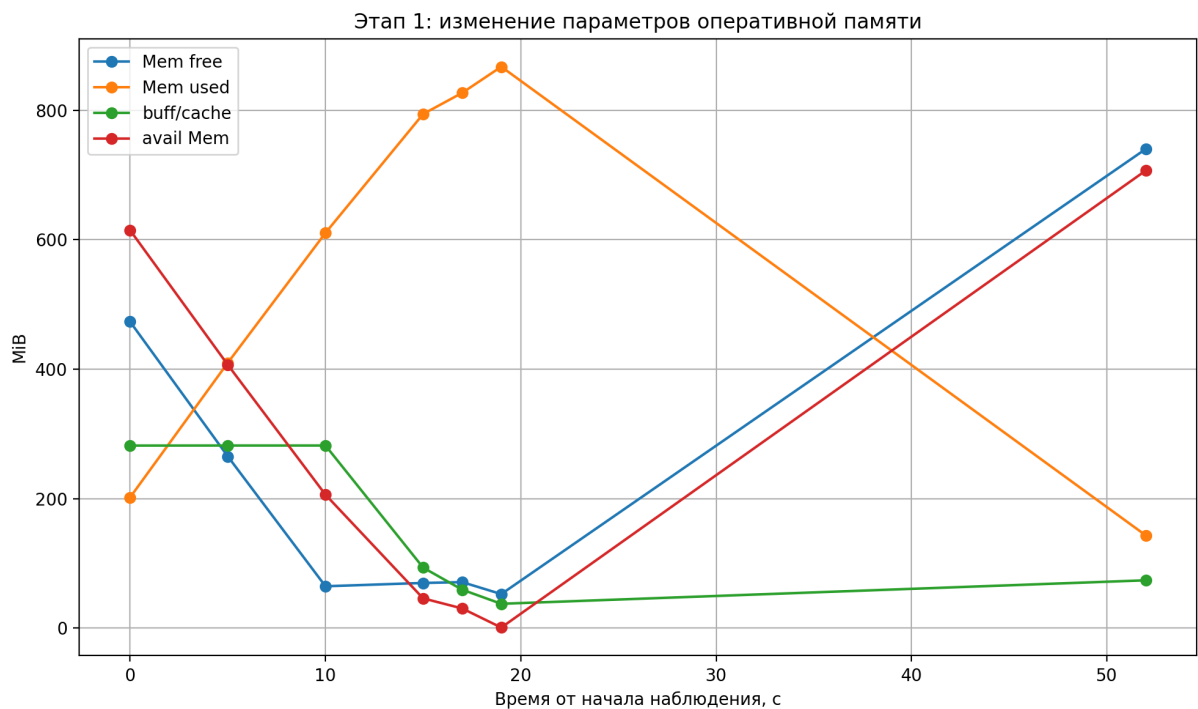


Рис. 2: Изменение параметров оперативной памяти на первом этапе

Вывод. Во время работы `mem.bash` свободная и доступная память уменьшались, а после завершения процесса память была освобождена.

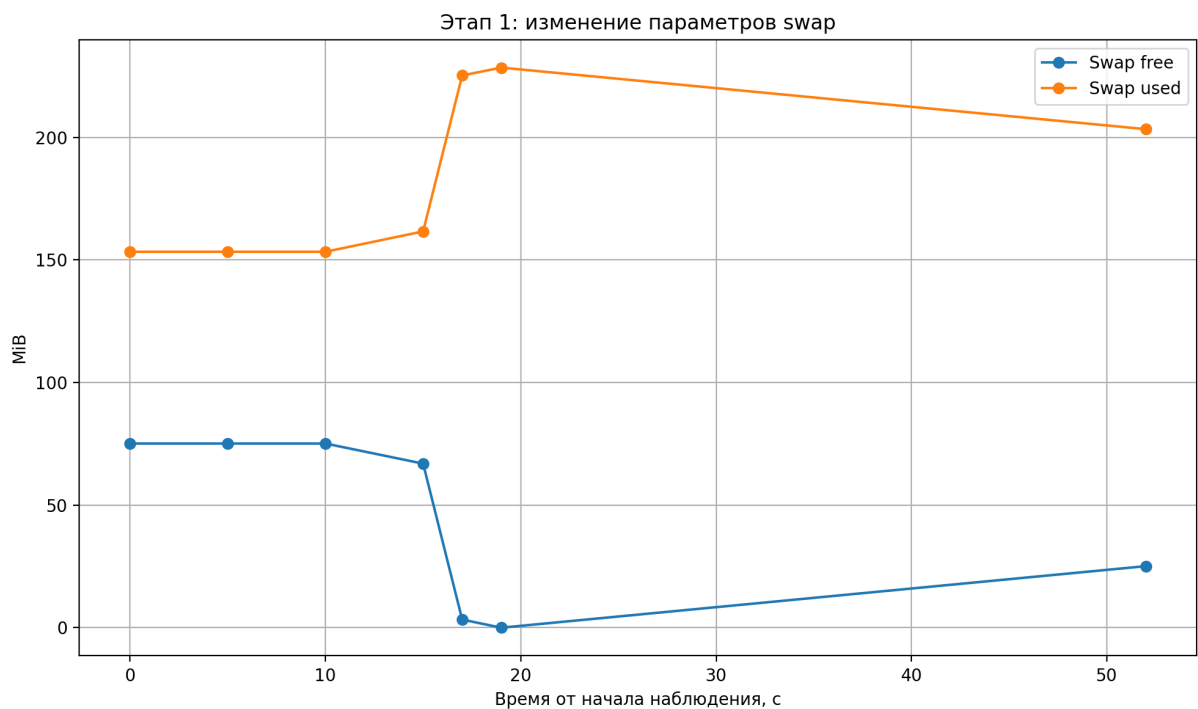


Рис. 3: Изменение параметров swap на первом этапе

Вывод. При приближении к аварийному завершению свободное пространство в swap уменьшилось почти до нуля.

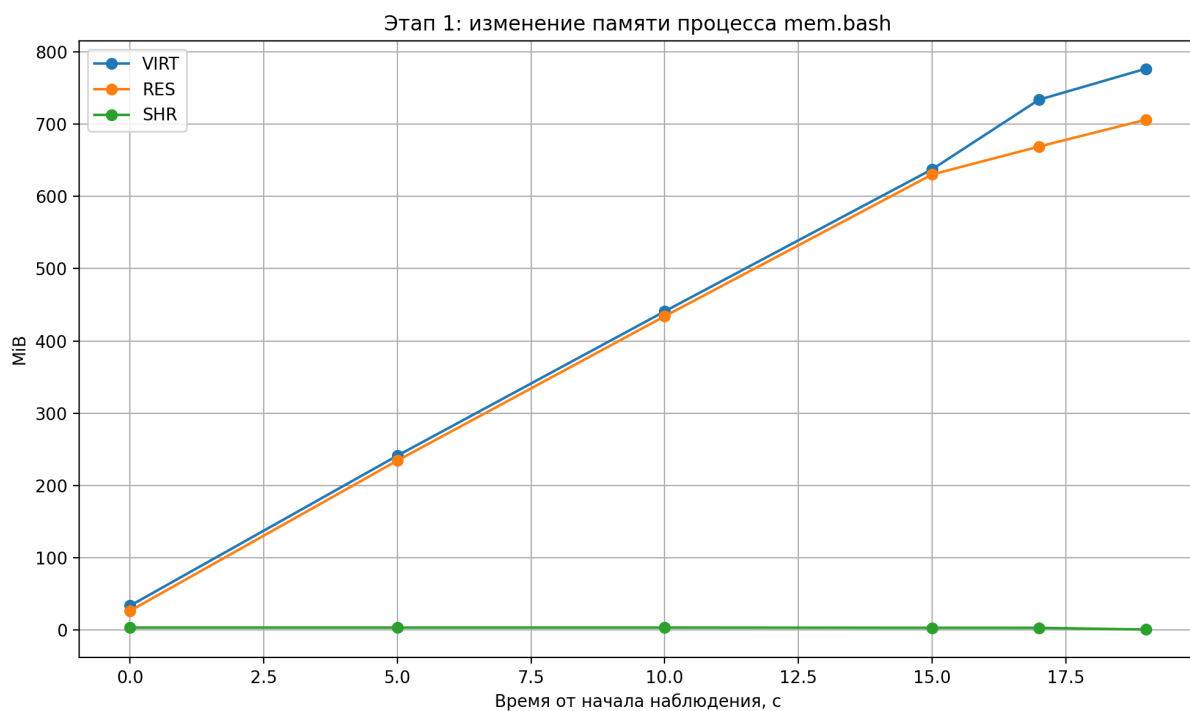


Рис. 4: Изменение памяти процесса `mem.bash` на первом этапе

Вывод. Значения `VIRT` и `RES` процесса `mem.bash` последовательно росли из-за увеличения массива.

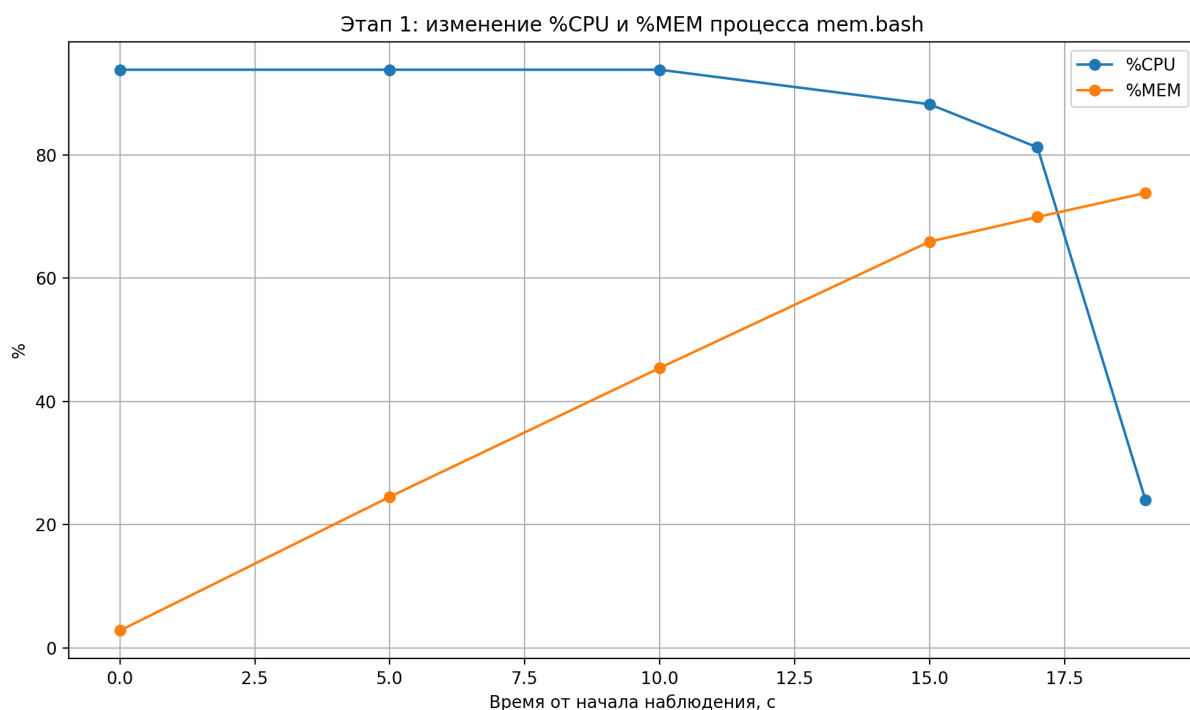


Рис. 5: Изменение %CPU и %MEM процесса `mem.bash` на первом этапе

Вывод. Доля используемой памяти `%MEM` росла, а загрузка CPU снизилась ближе к завершению из-за дефицита памяти и работы подкачки.

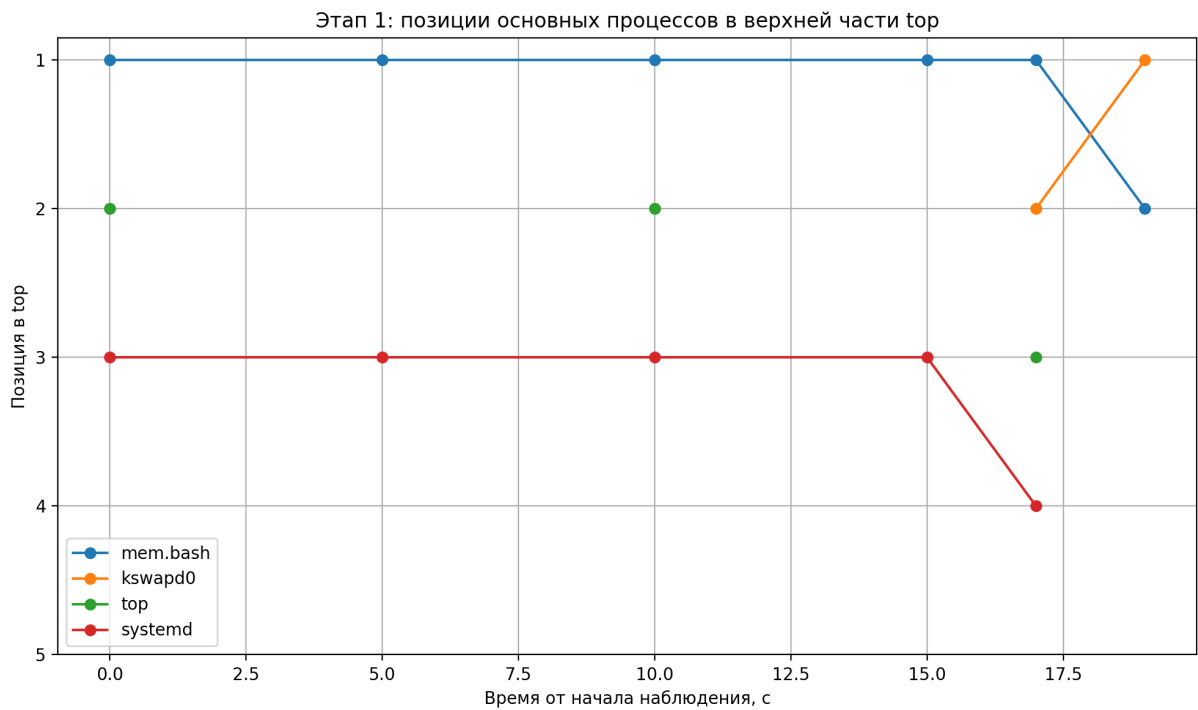


Рис. 6: Изменение позиций основных процессов в верхней части `top` на первом этапе

Вывод. Процесс `mem.bash` находился среди верхних процессов, а ближе к аварийному завершению выше него появился `kswapd0`.

3.3 Второй этап: два процесса `mem.bash` и `mem2.bash`

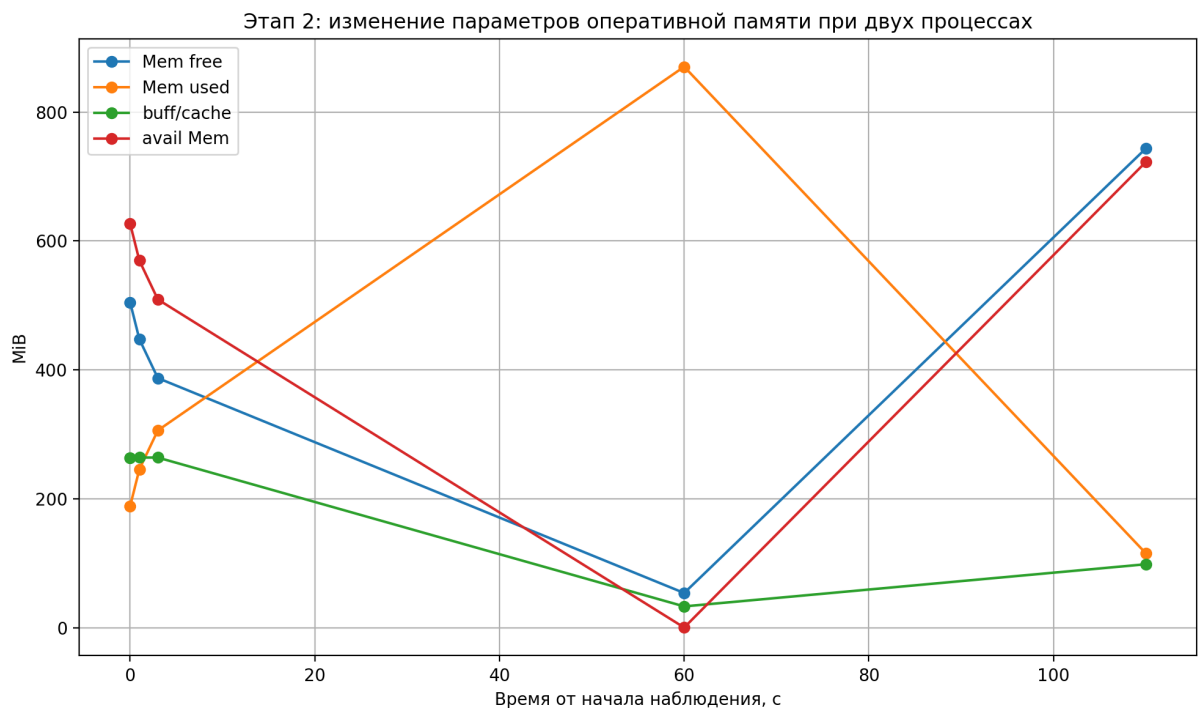


Рис. 7: Изменение параметров оперативной памяти при двух процессах

Вывод. При одновременной работе двух процессов доступная память уменьшалась быстрее, чем при одиночном запуске.

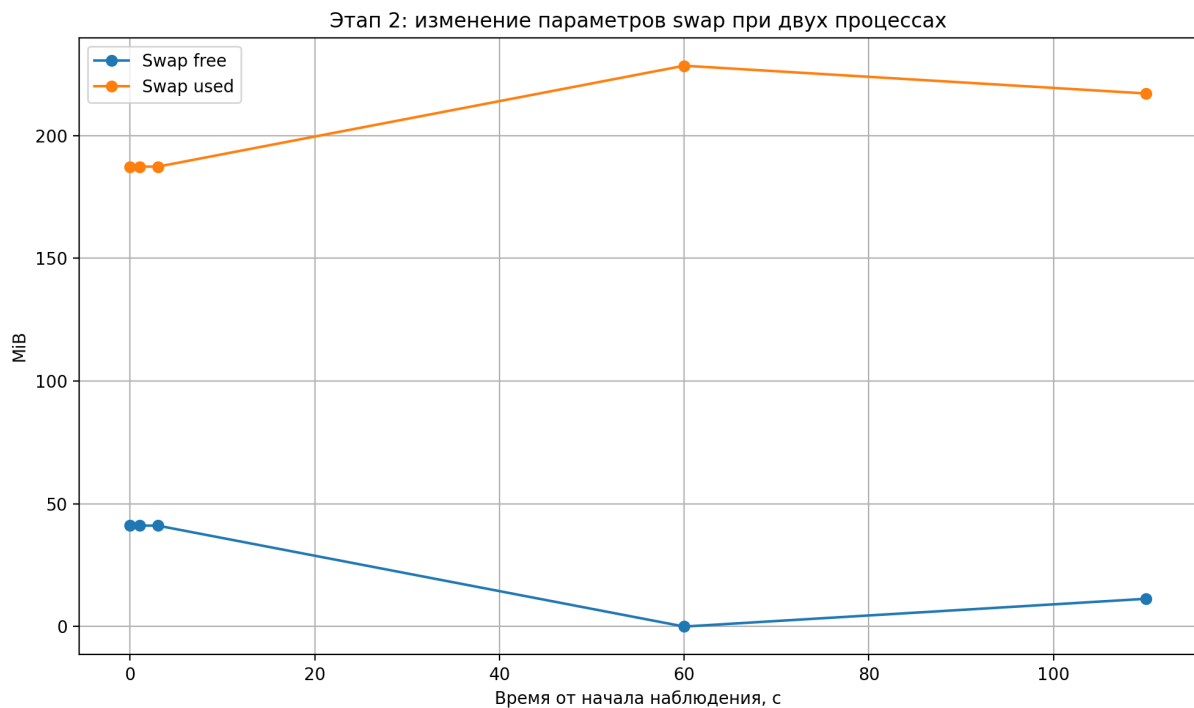


Рис. 8: Изменение параметров swar при двух процессах

Вывод. Во втором этапе swar был полностью использован, что указывает на сильный дефицит оперативной памяти.

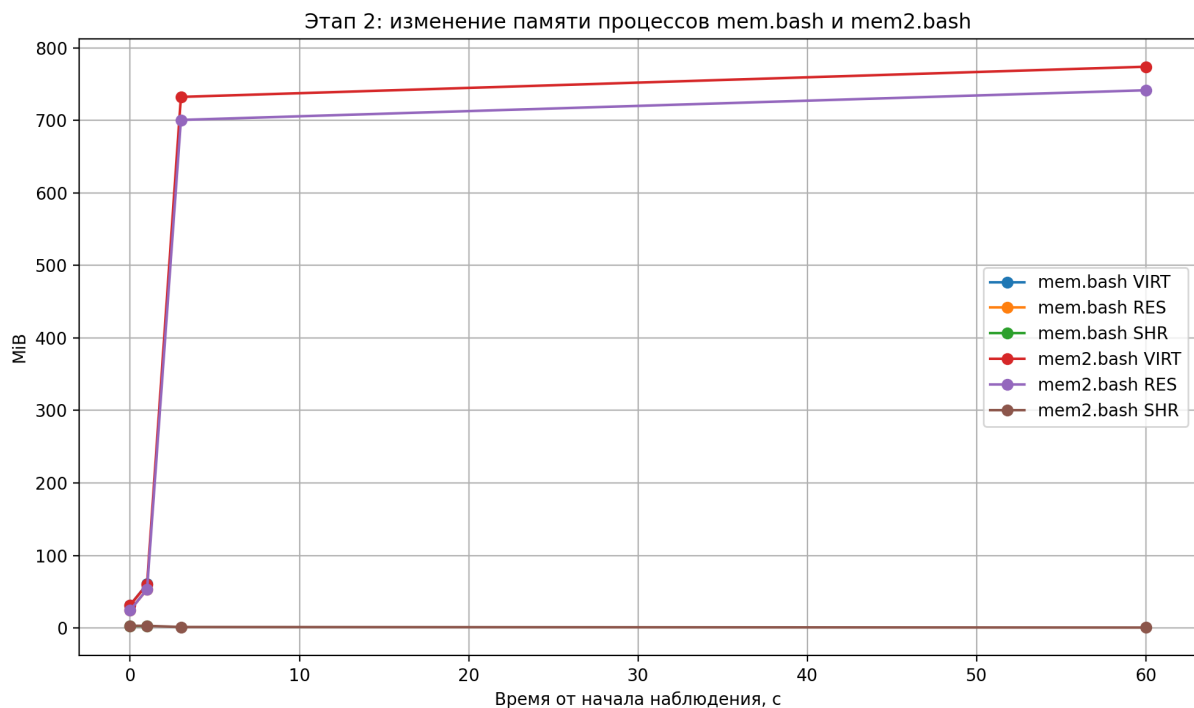


Рис. 9: Изменение памяти процессов mem.bash и mem2.bash

Вывод. После завершения mem.bash процесс mem2.bash продолжил увеличивать потреб-

ление памяти до аварийного завершения.

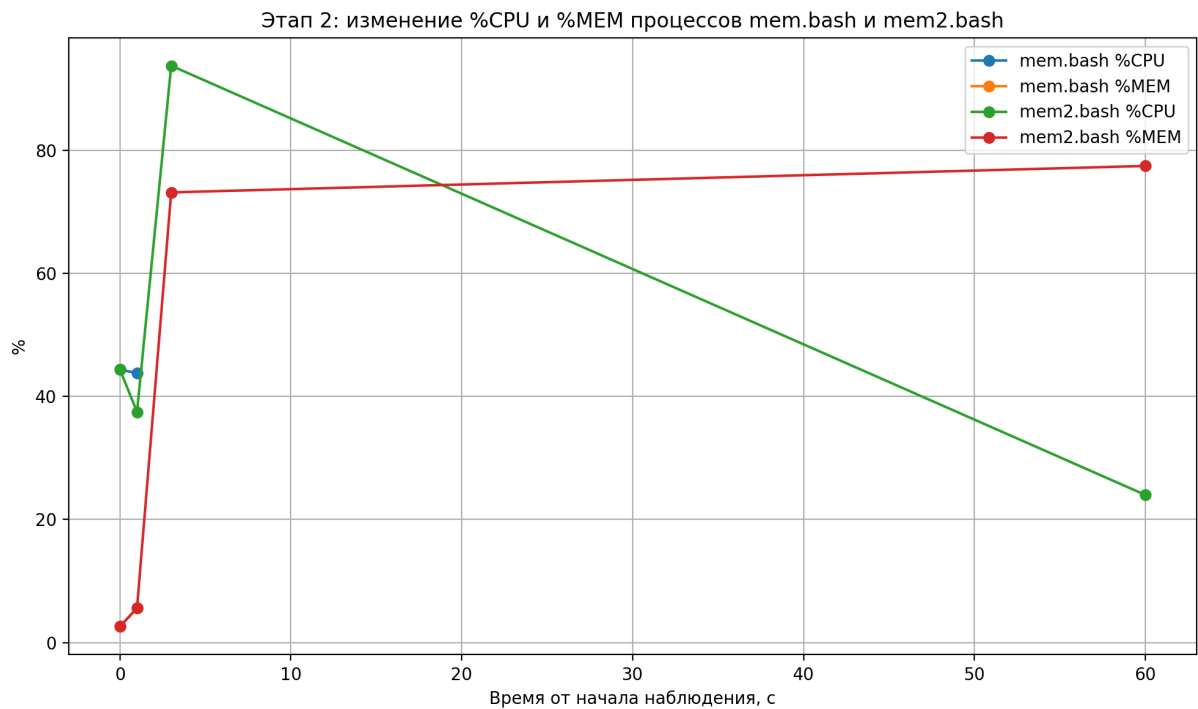


Рис. 10: Изменение %CPU и %MEM процессов `mem.bash` и `mem2.bash`

Вывод. Значение %MEM у `mem2.bash` выросло до максимума после завершения первого процесса.

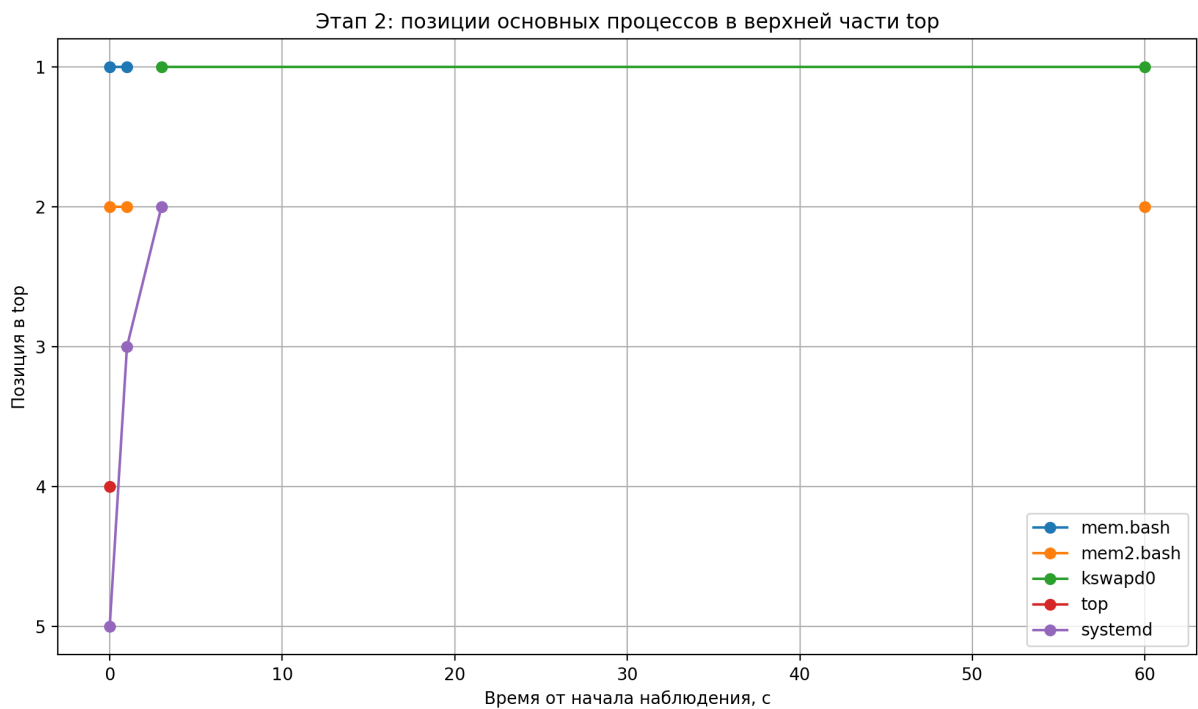


Рис. 11: Изменение позиций основных процессов в верхней части `top` на втором этапе

Вывод. При нехватке памяти процесс `kswapd0` поднялся в верхнюю часть списка процессов, так как система активно использовала подкачку.

3.4 Объяснение динамики изменения параметров

Скрипты `mem.bash` и `mem2.bash` непрерывно увеличивали массив, поэтому возрастали значения `VIRT`, `RES` и `%MEM`. Так как память в Linux выделяется страницами, рост массива приводил к выделению новых страниц памяти.

По мере роста потребления памяти уменьшались `Mem free` и `avail Mem`. При нехватке оперативной памяти система начала использовать `swap`, что видно по уменьшению `Swap free` и росту `Swap used`. Появление `kswapd0` связано с активной работой механизма подкачки.

После завершения процессов занятая ими память была освобождена, поэтому `Mem free` и `avail Mem` снова увеличились.

3.5 Объяснение пороговых величин

Пороговое значение размера массива — это последнее значение, записанное в `report.log` или `report2.log` перед аварийным завершением процесса. Так как запись выполнялась только каждый 100000-й шаг, это значение является приближенной нижней оценкой размера массива в момент завершения.

На первом этапе для процесса `mem.bash` было зафиксировано:

```
100000000
```

На втором этапе были зафиксированы значения:

```
report.log:
50000000
```

```
report2.log:
100000000
```

При запуске двух процессов `mem.bash` был завершен раньше, потому что оба процесса одновременно потребляли память и быстрее привели систему к нехватке ресурсов.

3.6 Параметры аварийного завершения процессов

В системном журнале были зафиксированы параметры процессов в момент их аварийного завершения.

Таблица 5: Параметры процессов в момент аварийного завершения

Этап	Процесс	PID	total-vm, KB	anon-rss, KB	file-rss, KB	pgtables, KB
Этап 1	mem.bash	1723068	800732	728064	0	1604
Этап 2	mem.bash	1733101	408560	381056	0	832
Этап 2	mem2.bash	1733102	804032	770052	0	1616

Параметр `total-vm` показывает общий объем виртуальной памяти процесса, а `anon-rss` показывает объем анонимной резидентной памяти, то есть памяти, фактически занятой данными процесса в оперативной памяти. Значения `file-rss` и `shmem-rss`, равные нулю, показывают, что основная причина потребления памяти была связана с ростом массива внутри процесса. Параметр `pgtables` показывает объем памяти, занятый таблицами страниц.

3.7 Общие выводы

Рост массива в Bash приводит к постепенному увеличению потребления виртуальной и резидентной памяти процесса.

При нехватке оперативной памяти система начинает активнее использовать swar, однако после исчерпания доступной памяти и пространства подкачки процесс завершается механизмом OOM Killer.

При запуске двух процессов суммарное потребление памяти растет быстрее, поэтому аварийное завершение наступает раньше для одного из процессов.

Появление kswapd0 среди верхних процессов подтверждает активную работу механизма подкачки при нехватке физической памяти.

Параметры системного журнала показывают, что аварийное завершение было вызвано именно ростом анонимной резидентной памяти, связанной с массивом процесса.

4 Эксперимент №2

4.1 Проверка работы при $N = 1000000$ и $K = 10$

были выбраны следующие значения:

$$N = 1000000$$

$$K = 10$$

Перед запуском и после завершения всех процессов было проверено количество записей об аварийном завершении `newmem.bash` в системном журнале:

```
karasik@vds2923400:~/os_lab5$ ./check_if_k_runs_stopped.sh
Before: 20
After: 20
```

Так как количество записей до и после запуска не изменилось, то новых аварийных завершений процесса `newmem.bash` не произошло.

Следовательно, при $N = 1000000$ и $K = 10$ все запуски `newmem.bash` завершились успешно.

При $K = 30$ новые процессы запускаются каждую секунду, при этом ранее запущенные процессы еще не успевают завершиться.

В результате в системе одновременно находится больше процессов `newmem.bash`. Каждый из них выделяет память под свой массив, поэтому суммарное потребление памяти значительно возрастает.

Когда суммарное потребление памяти превышает доступный объем оперативной памяти и пространства подкачки, ядро Linux запускает механизм OOM Killer. В результате часть процессов `newmem.bash` завершается аварийно.

4.2 Вывод

В ходе эксперимента было установлено, что при увеличении количества запусков до $K = 30$ часть процессов может завершаться аварийно, так как несколько экземпляров `newmem.bash` одновременно потребляют оперативную память.

Для предотвращения аварийных завершений было подобрано значение параметра:

$$N = 800000$$

При данном значении N процессы `newmem.bash` завершаются до того, как их суммарное потребление памяти приводит к исчерпанию доступной оперативной памяти и swap.